BOOKS & AUTHORS GRAPHQL API

Java and Spring Boot Interview Guide

A practical walkthrough of the actual Kotlin implementation

Stack	Focus
Kotlin 2.2.21	GraphQL schema and resolvers
Java 17	JPA / Hibernate / H2
Spring Boot 3.5.16	Batch relationship loading
Gradle 8.14.4	Interview preparation

Prepared 29 October 2026

Based on inspected repository source, tests, and project documentation.

Contents

1. Project overview **3**

Problem and goals . . . 3

Stack and purpose . . . 3

Implemented features . . . 3

Project structure . . . 3

2. Architecture and request flow **5**

Simple view . . . 5

Representative books query sequence . . . 5

Responsibilities and boundaries . . . 5

3. GraphQL fundamentals through this project **7**

GraphQL and REST . . . 7

SDL types and nullability . . . 7

Queries, mutations, and field selection . . . 7

Error and missing-record behavior . . . 7

4. Java concepts relevant to this Kotlin implementation **8**

Classes, objects, interfaces, and generics . . . 8

Collections and lambdas . . . 8

Nullability and exceptions . . . 8

ORM concepts . . . 8

5. Kotlin and Spring Boot **9**

Kotlin features used here . . . 9

Spring Boot and dependency injection . . . 9

Controller, service, and repository design . . . 9

6. JPA, Hibernate, and database design **10**

Entities and relationship . . . 10

Transactions and persistence context . . . 10

CRUD and integrity behavior . . . 10

7. Pagination and validation **11**

8. N+1 query problem **12**

What N+1 means . . . 12
The project's batching approach . . . 12
SQL regression evidence . . . 12

9. API walkthrough **13**
List paginated Books . . . 13
Fetch one Book . . . 13
List Authors . . . 13
Add a Book . . . 13
Update a Book . . . 13
Delete a Book . . . 14

10. Testing strategy and recorded results **15**
Test styles . . . 15
Results and command . . . 15

11. Run and demonstrate **16**
Prerequisites and start . . . 16

12. Interview questions and answers **17**
Java and Kotlin . . . 17
Spring and GraphQL . . . 17
Persistence and transactions . . . 17
Pagination, N+1, and tests . . . 18

13. Final interview cheat sheet **19**
Architecture at a glance . . . 19
Key classes . . . 19
Important annotations . . . 19
Five design decisions to remember . . . 19
Honest limitations . . . 19
Glossary . . . 19
Project references . . . 20

1. Project overview

Problem and goals

This project provides a small Books and Authors API through GraphQL. A client can read paginated books, fetch one book, list authors, and create, update, or delete books. Each book must refer to one existing author. The application demonstrates a Kotlin/JVM service built with Spring Boot, Spring for GraphQL, Spring Data JPA, Hibernate, and H2.

The work is also a learning and demonstration project: the schema is explicit, resolver methods are small, persistence behavior is testable, and the nested Author field is batched to avoid a per-book relationship query pattern. The user-approved mutation and pagination contracts are project choices; the supplied assignment material did not prescribe their exact signatures.

Stack and purpose

Java 17 / JVM — Runtime and compilation target. The project is Kotlin but runs on the JVM.
Kotlin 2.2.21 — Application and test language; null safety, concise data classes, collection operations.
Spring Boot 3.5.16 — Application startup, dependency injection, configuration, and framework auto-configuration.
Spring for GraphQL — SDL-based schema, GraphQL HTTP endpoint, resolver annotations, GraphiQL integration.
Spring Data JPA — Repository interfaces and pagination abstraction.
Hibernate / Jakarta Persistence — ORM implementation, entity mapping, lazy association, schema generation.
H2 — In-memory development and test database; demo profile seeds one Author.
Gradle Kotlin DSL 8.14.4 — Build and dependency configuration via `build.gradle.kts`; wrapper makes Gradle reproducible.
JUnit 5 / GraphQLTester — Integration testing of GraphQL operations and persistence behavior.
MockMvc — In-process HTTP checks for GraphQL, GraphiQL, Swagger UI, and OpenAPI metadata.
springdoc-openapi 2.10.0 — OpenAPI metadata and Swagger UI for HTTP REST paths; it does not describe GraphQL fields.

Implemented features

- `books(page, size)` uses repository-side pagination, zero-based pages, size bounds 1 through 100, and ascending Book ID ordering.
- `bookById(id)` returns a Book or `null` when no record exists; the missing-record response is tested without a GraphQL execution error.
- `authors` lists persisted Authors.
- `addBook`, `updateBook`, and `deleteBook` follow the approved project contracts. Book creation and reassignment use an existing Author ID. Book deletion retains its Author.
- `Book.author` is a required singular GraphQL field, persisted as a non-null foreign key and loaded in a batch.
- `/graphql` accepts GraphQL HTTP requests. `/graphiql` serves an in-browser editor. The demo profile seeds `demo-author-001`.
- `/v3/api-docs` and Swagger UI expose OpenAPI metadata. There are no REST controllers for the Books/Authors operations, so Swagger can show an empty paths section.

Project structure

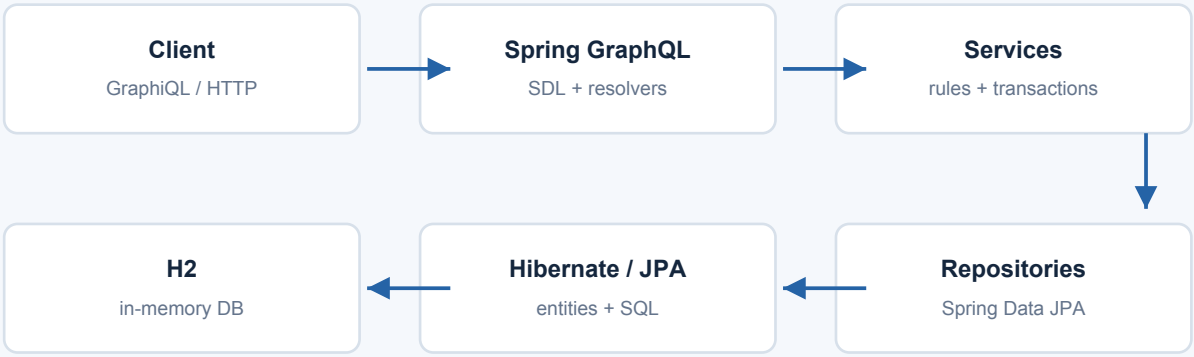
`src/main/resources/graphql/schema.graphqls` — Public GraphQL types, inputs, operations, and nullability.
`src/main/kotlin/com/example/booksapi/book/BooksQueryController.kt` — Root query resolvers and the batched `Book.author` resolver.
`src/main/kotlin/com/example/booksapi/book/BookMutationController.kt` — Thin mutation resolver methods.
`src/main/kotlin/com/example/booksapi/book/BookQueryService.kt` — Read operations, pagination validation, and read-only transaction boundary.
`src/main/kotlin/com/example/booksapi/book/BookMutationService.kt` — Transactional create/update/delete rules.

`src/main/kotlin/com/example/booksapi/book/Book.kt` — JPA Book entity and required Author association.
`src/main/kotlin/com/example/booksapi/author/Author.kt` — JPA Author entity.
`src/main/kotlin/com/example/booksapi/book/BookRepository.kt` — Spring Data Book persistence and paged access.
`src/main/kotlin/com/example/booksapi/author/AuthorRepository.kt` — Spring Data Author persistence.
`src/main/resources/application.yml` — H2, Hibernate, and GraphiQL configuration.
`src/main/resources/application-demo.yml`, `demo-data.sql` — Demo-only initialization and deterministic Author seed.
`src/test/kotlin/com/example/booksapi/` — GraphQL, HTTP, persistence, pagination, SQL-count, and profile tests.

2. Architecture and request flow

Simple view

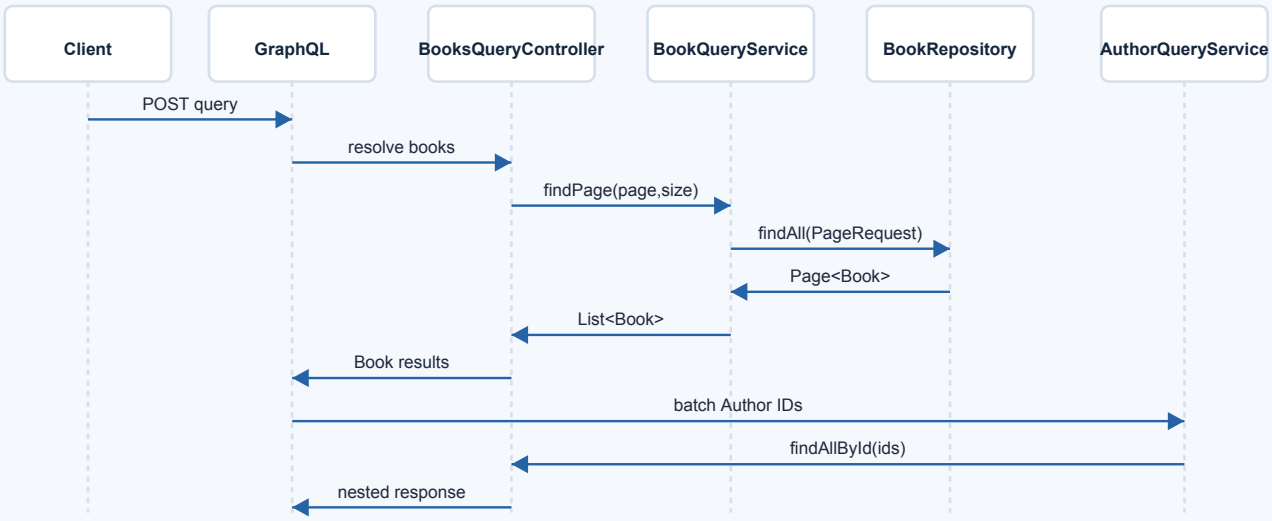
A client sends a GraphQL document to the Spring GraphQL HTTP endpoint. Spring validates it against the SDL, selects the matching resolver, and invokes application services. Services use Spring Data repositories. Hibernate translates repository operations into SQL against H2. For a nested `author` selection, the `@BatchMapping` resolver loads Authors for a group of Books and maps each result to its Book.



Nested Book.author resolution is batched by @BatchMapping before the Author repository lookup.

Representative `books` query sequence

The example below requests nested data, so Spring resolves both `books` and each selected `author` field. The root resolver returns Book entities; the batch resolver gathers their Author IDs and makes one repository lookup for the group in the tested scenario.



The batch resolver is invoked for selected nested fields; SQL capture counts database statements, not resolver calls.

Responsibilities and boundaries

- Schema and resolver layer: binds operation names and arguments to Kotlin methods. It should not own database transaction policy or duplicate business rules.

- Service layer: validates page bounds, applies mutation rules, and defines transactional work.
- Repository layer: exposes CRUD and pageable access without hand-written SQL in this project.
- Persistence layer: entities and Hibernate mappings describe stored state and the required foreign key.
- Database: H2 is volatile in-memory storage. Restarting the application resets data; demo startup then seeds the one configured Author.

3. GraphQL fundamentals through this project

GraphQL and REST

REST commonly presents resources through multiple HTTP paths and fixed response representations. GraphQL exposes a schema at an endpoint and lets a client select fields in an operation. This application uses the `/graphql` endpoint and SDL-defined operations; the response shape follows the selection set. GraphQL does not automatically make backend work efficient: resolvers still need deliberate data loading, which is why this project uses `@BatchMapping` for `Book.author`.

SDL types and nullability

The actual schema is:

GRAPHQL

```
type Query {
  books(page: Int! = 0, size: Int! = 20): [Book!]!
  bookById(id: ID!): Book
  authors: [Author!]!
}

type Mutation {
  addBook(input: BookInput!): Book!
  updateBook(id: ID!, input: UpdateBookInput!): Book!
  deleteBook(id: ID!): Boolean!
}

type Book { id: ID!, title: String!, author: Author! }
type Author { id: ID!, name: String! }
input BookInput { title: String!, authorId: ID! }
input UpdateBookInput { title: String, authorId: ID }
```

`!` means a field or argument cannot be null. `[Book!]!` means the list itself and each item are non-null. `Book` on the `bookById` field is nullable, so absence is represented as `null`. Input object types are used for mutation arguments; output object types describe data clients can select.

Queries, mutations, and field selection

A query reads data; a mutation requests a state change. A client selects only the fields it needs. Selecting `author { id name }` under a Book requests nested Author data in the same GraphQL operation. This proves relationship functionality. It does not, by itself, prove the number of database queries; that requires SQL measurement.

Error and missing-record behavior

- `bookById` returns `null` for an unknown ID. The current GraphQLTester test asserts the null value and no execution errors.
- Invalid page/size arguments are rejected by service validation; the response has GraphQL errors.
- Mutations with missing Books or unknown Authors fail through standard Spring for GraphQL error handling. Tests ensure messages do not expose the service exception text, Java class name, application package, or stack frame.
- The assignment screenshots do not define a detailed error payload contract. This project intentionally does not add a custom error-code framework.

4. Java concepts relevant to this Kotlin implementation

Classes, objects, interfaces, and generics

`Book` and `Author` are Kotlin classes that represent entity instances. The `BookRepository` is an interface parameterized as `JpaRepository<Book, String>`: its entity type is `Book` and its ID type is `String`. Java developers will recognize the same generic interface and interface-based repository pattern. Spring creates the repository implementation at runtime and injects it where required.

Kotlin class properties such as `var title: String` compile to JVM fields/accessors with Kotlin metadata. They are not Java source syntax. The entity classes are `open` and provide a no-argument constructor, which supports Hibernate proxying and entity construction.

Collections and lambdas

The code uses `List<Book>`, `Iterable<String>`, and `Map<Book, Author>` as typed collections. Kotlin lambdas and collection operators include `map`, `associateBy`, `associateWith`, and `mapIndexed`. For example, the batch resolver maps Books to IDs, indexes returned Authors by ID, and assembles a `Map<Book, Author>`. These are Kotlin standard-library operations, similar in purpose to Java Streams but not Java Stream code.

Nullability and exceptions

Kotlin `Book?` explicitly allows null; `Book` does not. The query service uses `orElse(null)` and returns `Book?`. `lateinit var author: Author` is initialized by entity construction/persistence and is declared non-null; it must be initialized before access. Kotlin/JVM exceptions are not checked in Java's checked-exception sense. `require` and `IllegalArgumentException` communicate invalid input or missing references to the framework's GraphQL error handling.

ORM concepts

Object-relational mapping connects object properties to database columns and relationships. `@Entity`, `@Id`, `@Column`, and `@ManyToOne` are Jakarta Persistence annotations. Hibernate is the ORM implementation in this stack. The entity object model and foreign-key constraint work together: application rules produce useful errors, while the database constraint protects persisted integrity.

5. Kotlin and Spring Boot

Kotlin features used here

- `val` is a read-only reference; `var` can be reassigned. It does not mean the object referenced by a `val` is deeply immutable.
- `String?` is nullable; `String` is non-null unless platform interop weakens the guarantee.
- `BookInput` and `UpdateBookInput` are Kotlin data classes. The compiler generates value-oriented methods such as `equals`, `hashCode`, `toString`, and `copy`.
- `UpdateBookInput` uses Spring GraphQL `ArgumentValue<String?>`. This keeps “argument omitted” distinct from “argument explicitly provided as null”; a nullable property alone would not retain that distinction reliably.
- Constructor parameters on `@Service` and `@Controller` classes are constructor injection. There are no field `@Autowired` injections in production classes.

Spring Boot and dependency injection

`@SpringBootApplication` combines application configuration, component scanning, and auto-configuration. `runApplication<BooksAuthorsApplication>` starts the Spring application context. Component annotations register controllers and services as beans; Spring resolves their constructor dependencies and manages their lifecycle. Spring Boot auto-configuration uses the classpath and properties to configure web handling, GraphQL, JPA, Hibernate, the H2 datasource, and repository infrastructure.

The configuration is deliberately small. `spring.graphql.graphiql.enabled: true` enables the GraphiQL page supplied by the Spring GraphQL setup. H2 is configured with a named in-memory URL and Hibernate `ddl-auto: create-drop`. The `demo` profile defers SQL initialization until Hibernate has created the schema and loads `demo-data.sql`. The default profile does not seed an Author.

Spring for GraphQL discovers schema resources under `src/main/resources/graphql/` and maps annotated resolver methods to schema fields. The HTTP endpoint remains `/graphql`; GraphiQL is a browser client that submits operations to it.

Controller, service, and repository design

Query and mutation controllers adapt GraphQL input/output to application services. The mutation controller contains no persistence algorithm; the mutation service owns the transactional logic. The repository interfaces are injected by Spring Data. This separation supports focused tests and prevents GraphQL transport details from becoming database access code.

6. JPA, Hibernate, and database design

Entities and relationship

`Book` has a string ID, title, and required `Author`. `Author` has a string ID and name. Each Book references exactly one Author; an Author can be referenced by multiple Books. These cardinality and mutation choices are user-approved project design decisions, not screenshot-prescribed API details.

In `Book.kt`, the association is `@ManyToOne(fetch = FetchType.LAZY, optional = false)` with `@JoinColumn(name = "author_id", nullable = false)`. This is a many-books-to-one-author mapping from the Book side. `optional = false` expresses the required relationship to JPA; `nullable = false` expresses the non-null join column. The schema also declares `Book.author: Author!`. No remove cascade is configured.

A lazy association allows Hibernate to defer loading the Author when a Book is first loaded. The batch resolver collects Author IDs and calls `AuthorQueryService.findAllByIds`. The tested SQL measurement records the result of that path. Avoid claiming that `LAZY` alone solves N+1; batching is the mechanism used here.

Transactions and persistence context

The query service methods use `@Transactional(readOnly = true)`. Mutation service methods use `@Transactional`, so a mutation's repository work executes within a transaction managed by Spring. A JPA persistence context tracks entity state during a transaction; when a managed Book's title or Author reference changes, Hibernate detects and writes changes as part of transaction completion. `save` is called explicitly in the service as well.

CRUD and integrity behavior

- Add: look up the supplied Author ID, fail if not found, then save a Book with a generated UUID string ID. The API has no `addAuthor` operation and does not create Authors implicitly.
- Update: find the Book or fail. Omitted title/Author fields are preserved. Supplied valid values are applied. Explicit null title or Author ID is rejected; an Author cannot be cleared.
- Delete: find the Book or fail, delete only that Book, and return `true`. The Author remains even if no Book references it.
- A non-null foreign key and database referential integrity backstop application-level association rules.

The code uses H2 in-memory storage with `create-drop`. Data is not durable across process restarts. The `demo` profile seeds `demo-author-001` / `Demo Author`; default startup has no seed. The SQL uses H2 `MERGE` keyed by ID, and tests verify that reapplying it does not duplicate the Author.

7. Pagination and validation

The project-selected contract is `books(page: Int! = 0, size: Int! = 20): [Book!]!`. It is provisional: the assignment requires pagination but does not prescribe these argument names, defaults, or response shape.

`BookQueryService.findPage` enforces `page >= 0` and `size in 1..100`. It constructs a Spring Data `PageRequest` sorted ascending by `id` and returns the page content. This performs database-side pagination rather than loading every Book and slicing in memory. Page numbering starts at zero. A valid page beyond the last result returns an empty list.

Tests cover first and next pages, stable order, adjacent-page uniqueness, beyond-end behavior, negative page, zero and negative sizes, size above 100, and the maximum 32-bit positive GraphQL `Int` as size. Invalid inputs are asserted to produce GraphQL errors without assuming an exact error message. The default arguments in SDL allow `books { id title }` without explicit pagination values.

8. N+1 query problem

What N+1 means

Suppose one SQL query loads N Books. If resolving each Book's Author then runs an additional SQL query, the total is approximately `1 + N`. For 20 Books that can mean 21 selects instead of a root query plus a batched relationship query. GraphQL resolver calls per object are not themselves database queries; count SQL statements to establish database behavior.

The project's batching approach

`BooksQueryController.bookAuthors` is annotated `@BatchMapping(typeName = "Book", field = "author")`. It receives a group of Books, extracts `book.author.id`, loads Authors with `findAllByIds`, indexes them by ID, and returns a map from each Book to its Author. This avoids the naive “query Author separately for each Book” resolver shape in the tested GraphQL path.

SQL regression evidence

The test-only `SqlCaptureStatementInspector` implements Hibernate's `StatementInspector`. A `ThreadLocal` list records SQL only while `measure { ... }` wraps GraphQL execution. Fixture insertion/flush and cleanup occur outside that capture. The pagination test class classifies SELECTs by table and reports root Book fetches separately from Author queries. Other statements are counted separately and expected to be zero in the measured operation.

The recorded comparison uses the same nested query and H2 configuration with one distinct Author per Book:

5 — 1 | 1
20 — 1 | 1

Thus `deltaBooks = 20 - 5 = 15`, `deltaAuthorQueries = 1 - 1 = 0`, and the ratio is `0 / 15 = 0`. The test's actual pass criterion is fewer than 0.5 Author-query statements per additional returned Book; a ratio at or above 0.5 fails. This is practical evidence for the tested query and fixture, not proof about every possible workload or N+1 case. The recorded numbers are not a fresh run for this document.

9. API walkthrough

Use GraphiQL at `http://localhost:8080/graphiql` or POST GraphQL documents to `http://localhost:8080/graphql`. For mutations, start with the demo profile and query `authors` to obtain the existing ID.

List paginated Books

GRAPHQL

```
query {
  books(page: 0, size: 5) {
    id
    title
    author { id name }
  }
}
```

Expected: a list of Books with nested Authors. Default page and size are 0 and 20. Page size must be between 1 and 100.

Fetch one Book

GRAPHQL

```
query {
  bookById(id: "<book-id>") {
    id
    title
    author { id name }
  }
}
```

Expected: one Book or `null` for an unknown ID, without an execution error for that missing record.

List Authors

GRAPHQL

```
query {
  authors { id name }
}
```

In the demo profile, the initial result includes `demo-author-001` / `Demo Author`.

Add a Book

GRAPHQL

```
mutation {
  addBook(input: { title: "Interview Notes", authorId: "demo-author-001" }) {
    id
    title
    author { id name }
  }
}
```

Expected: a generated ID and nested existing Author. An unknown Author ID produces a GraphQL error; no Author is created implicitly.

Update a Book

GRAPHQL

```
mutation {
  updateBook(id: "<book-id>", input: { title: "Interview Notes - revised" }) {
    id
    title
    author { id name }
  }
}
```

Omitted fields are preserved. To change the Author, provide a valid `authorId`. Explicit `authorId: null` is rejected; it cannot clear the required relationship. Missing Books and unknown Authors produce GraphQL errors.

Delete a Book

GRAPHQL

```
mutation {
  deleteBook(id: "<book-id>")
}
```

Expected: `true` after successful deletion. The associated Author remains. An unknown Book ID produces a GraphQL error.

10. Testing strategy and recorded results

Test styles

- GraphQLTester: uses GraphQL documents and variables against a running Spring application context configured by `@SpringBootTest` and `@AutoConfigureGraphQlTester`. It asserts GraphQL data paths and error presence.
- MockMvc: drives the HTTP layer without opening a network port. `OpenApiHttpTests` check `/v3/api-docs`, Swagger UI, GraphiQL HTML, POST `/graphql`, and an HTTP mutation lifecycle.
- JPA repository assertions: verify persisted counts, relationships, deletion behavior, and foreign-key rejection in H2.
- SQL regression test: the test-only StatementInspector captures query SQL while the GraphQL operation executes.
- Profiles: default-profile test confirms the demo seed is absent; demo-profile tests confirm seed presence and idempotence.

Test classes include `BooksQueryTests`, `BooksMutationTests`, `BooksPaginationAndNPlusOneTests`, `BooksAuthorsApplicationTests`, `DemoProfileSeedTests`, and `OpenApiHttpTests`. There is no `@GraphQlTest` slice in the current suite; tests use full application contexts and GraphQLTester/MockMvc.

Results and command

Run the suite with:

SH

```
./gradlew clean test --console=plain
```

The latest successful full run before this guide was generated reported `BUILD SUCCESSFUL`: 29 tests, 0 failures, 0 errors, and 0 skipped. This is recorded run evidence, not a test run performed while writing this guide. Earlier milestone totals (M6: 21; M7: 23; GraphiQL change: 28) are historical snapshots and should not be mistaken for the current suite total.

11. Run and demonstrate

Prerequisites and start

Use the installed Java 17 runtime and the included wrapper (Gradle 8.14.4); system Gradle is not required. Check Java with `java -version`. From the repository root:

SH

```
./gradlew clean test --console=plain
./gradlew bootRun --args='--spring.profiles.active=demo' --console=plain
```

The second command activates the demo profile. It creates the H2 schema and seeds the demo Author. Open `http://localhost:8080/graphiql`, query `authors`, and then use the returned ID in the mutation examples. The GraphiQL page's module, stylesheet, and worker imports use `esm.sh`; a browser internet connection is needed for those UI assets, while the application endpoint remains local.

Execute the examples in Section 9 in order: list Authors, add a Book, update its title or Author, fetch it, then delete it and query Authors again to see that the Author remains. Stop the foreground application with Ctrl+C. Because H2 is in-memory, restarting resets Books; the demo profile seeds the Author again.

Swagger UI is at `http://localhost:8080/swagger-ui/index.html` and JSON metadata at `http://localhost:8080/v3/api-docs`. These endpoints do not document the GraphQL schema or operations. No REST Books/Authors controllers are present, so an empty OpenAPI `paths` section is expected.

12. Interview questions and answers

Java and Kotlin

1. Is this application written in Java? No. Application code is Kotlin targeting the JVM. Java concepts are relevant to its runtime and Spring ecosystem, but the examples must be described as Kotlin.

2. What is the difference between a class and an object here? `Book` is a class definition; a loaded or constructed Book is an object/entity instance. `Book` instances are mapped to rows by JPA.

3. What does `JpaRepository<Book, String>` mean? It is a generic repository for Book entities whose ID type is String. Spring Data creates the implementation.

4. What is Kotlin null safety? A `String` is non-null by type, while `String?` may hold null. `Book?` models the nullable `bookById` result. This moves many null checks into compile-time type checking.

5. What does `val` guarantee? The reference cannot be reassigned. It does not guarantee that a referenced mutable object cannot change.

6. Why is `UpdateBookInput` a data class? It is a compact value carrier. Kotlin generates equality, hash, string, and copy methods. It uses `ArgumentValue` properties so omitted input fields are distinguishable from explicit null.

7. Are Kotlin lambdas Java Streams? No. The project uses Kotlin collection operators such as `map` and `associateBy`. They are analogous in purpose to some Stream operations but use Kotlin standard-library APIs.

Spring and GraphQL

8. What does Spring Boot add? It provides application startup, auto-configuration, embedded web runtime integration, externalized properties, and convenient component wiring based on the dependencies present.

9. What is dependency injection in this code? Spring constructs controller/service beans and supplies their constructor dependencies, such as `BookRepository`, rather than those classes creating dependencies themselves.

10. Why separate controller, service, and repository? The resolver adapts the GraphQL operation, the service owns application rules and transaction boundaries, and the repository handles persistence. Each layer has a focused role.

11. What does `@QueryMapping` do? It maps a Kotlin method to a root query field in the SDL. `@MutationMapping` does the same for a mutation.

12. What is SDL? Schema Definition Language declares GraphQL object/input types, operations, arguments, lists, and nullability. Spring for GraphQL validates operations against this schema.

13. How does GraphQL differ from REST in this project? The API exposes a GraphQL endpoint and schema; clients select response fields in a document. This project does not implement REST CRUD routes for Books or Authors.

14. Why can a client ask for nested Author data? `Book.author: Author!` links the Book output type to the Author object type, and the field resolver supplies the associated Author.

15. What does `[Book!]!` mean? The returned collection cannot be null, and no element in the list may be null. The list may be empty.

16. What happens for an unknown `bookById`? The query service returns null. GraphQL returns `data.bookById = null`; the current test checks that this expected null has no execution error.

17. Does Swagger document the GraphQL schema? No. springdoc provides OpenAPI metadata and Swagger UI for REST paths; it does not automatically translate GraphQL SDL into OpenAPI operations. An empty REST paths list is expected here.

Persistence and transactions

18. What is an entity? A class marked `@Entity` whose instances are managed by JPA and correspond to database rows. `Book` and `Author` are entities.

19. What does `@ManyToOne` mean here? Many Books may point to one Author. The foreign key is stored on the Book row as `author_id`.

20. Why use `FetchType.LAZY`? It asks the persistence provider to defer association loading until needed. Lazy loading does not automatically prevent N+1; this project batches Author resolution separately.

21. Why is the relationship non-optional? The approved design says every Book has an Author. JPA marks it `optional = false`, the join column is non-null, and GraphQL declares `Author!`.

22. What does `@Transactional` do in the mutation service? Spring runs the method's persistence work in a transaction. On normal completion changes commit; on qualifying failures Spring rolls back according to its transaction rules.

23. Why does deleting a Book retain the Author? The service deletes only the Book, and the mapping has no cascade-remove behavior. An Author may be shared by multiple Books.

24. What is the role of H2? It is the project's in-memory relational database for development and tests. The chosen URL and `create-drop` schema mode make data volatile across application restarts.

25. How does invalid Author input fail? The service looks up the supplied Author ID and throws if missing. Spring for GraphQL reports a GraphQL error; the tests check that internal exception details are not leaked.

Pagination, N+1, and tests

26. How is pagination implemented? The service validates page and size and calls the repository with a `PageRequest`, sorted by ID ascending. This delegates slicing to the database.

27. What is N+1? One query loads a list of N parent rows, followed by one additional query for each parent's child/relationship. Query counts, not resolver method counts, establish whether SQL behaves that way.

28. How does `@BatchMapping` help? It resolves a relationship for a group of source objects and lets the implementation load the related Authors together. The current method queries by the group's IDs and maps Authors back to Books.

29. How is N+1 behavior regression-tested? A Hibernate `StatementInspector` captures SQL only during GraphQL execution. The test separates Book and Author selects at different result sizes and requires fewer than 0.5 added Author statements per added Book.

30. What is GraphQLTester? A Spring GraphQL testing API that submits GraphQL documents and supports assertions over response data and errors. This suite uses it with a full Spring application context.

31. Why test with MockMvc too? GraphQLTester focuses on GraphQL execution. MockMvc verifies actual Spring MVC HTTP routes and responses without opening a network port.

32. What is the security posture? No Spring Security dependency, filter chain, authentication, or authorization is configured. This is a learning/demo application and should not be exposed as a production service without appropriate access controls and deployment protections.

33. Is CORS or CSRF configured? No explicit application CORS or Spring Security CSRF policy is configured. With no Spring Security dependency, do not describe the app as having security rules; production browser and mutation protections need an explicit deployment design.

34. What data is seeded by default? None. Only the `demo` profile loads `demo-data.sql`, which inserts the deterministic `Demo Author` record. Tests verify default-profile absence and demo-profile presence/idempotence.

35. What should be improved for production? Add a deliberate authentication and authorization design, external durable database and migration strategy, operational configuration, and security review. Those are future improvements, not implemented features.

13. Final interview cheat sheet

Architecture at a glance

`GraphQL document -> Spring for GraphQL resolver -> service -> Spring Data repository -> Hibernate/JPA -> H2`

For nested Book authors: `@BatchMapping -> findAllByIds -> Map<Book, Author>`.

Key classes

`BooksAuthorsApplication` — Spring Boot entry point.
`BooksQueryController` — `books`, `bookById`, `authors`, and batched `Book.author` resolvers.
`BookMutationController` — GraphQL mutation adapter.
`BookQueryService` — Read operations, page bounds, pageable fetch, read-only transaction.
`BookMutationService` — Transactional mutation rules and Author validation.
`Book` / `Author` — JPA entities; Book owns required many-to-one association.
`BookRepository` / `AuthorRepository` — Spring Data persistence interfaces.
`SqlCaptureStatementInspector` — Test-only SQL capture around GraphQL execution.

Important annotations

`@SpringBootApplication` — Boot configuration, component scanning, and auto-configuration entry point.
`@Controller` — Registers GraphQL resolver classes as Spring components.
`@QueryMapping` / `@MutationMapping` — Binds methods to root SDL fields.
`@BatchMapping` — Resolves `Book.author` for a group of Books.
`@Service` — Registers application service components.
`@Transactional` — Defines mutation transaction scope; `readOnly = true` marks query service reads.
`@Entity`, `@Id`, `@ManyToOne`, `@JoinColumn` — JPA entity, key, relationship, and foreign-key mapping.

Five design decisions to remember

1. Each Book has exactly one required Author; an Author may be shared by Books.
2. Book stores a non-null foreign key to Author; deletion of Book does not cascade to Author.
3. Mutations reference existing Authors only. Update omission preserves fields; explicit null cannot clear the required Author.
4. The pagination signature and bounds are a provisional project contract, not assignment-prescribed details.
5. `@BatchMapping` plus SQL-count regression evidence addresses relationship loading; nested data alone is not proof of query efficiency.

Honest limitations

- H2 is in-memory and not durable.
- Demo Author provisioning requires the explicit `demo` profile.
- GraphiQL shell is served locally but its UI imports external `esm.sh` assets.
- Swagger/OpenAPI does not document GraphQL operations; no REST Books/Authors controllers exist.
- There is no authentication/authorization or explicit CORS/CSRF security setup.
- The SQL ratio threshold is practical regression evidence for the tested fixtures, not a universal performance guarantee.

Glossary

API — A defined interface through which software clients request functionality.

Bean — An object created and managed by the Spring application context.
Dependency injection — Supplying an object's collaborators from a container rather than constructing them inside it.
Entity — A JPA-managed object associated with a relational table row.
Foreign key — A database column constraint that references another table's key.
GraphQL — A typed API query language and execution system with client-selected fields.
Hibernate — The ORM implementation used by JPA in this project.
Input type — GraphQL type used to pass structured arguments into operations.
JPA — Jakarta Persistence API, the standard mapping and persistence contract used here.
Lazy loading — Deferred loading of a mapped association until the data is accessed.
Mutation — GraphQL operation used to change application state.
N+1 — A query pattern with one root query followed by one relationship query per result row.
Non-null (`!`) — GraphQL marker that a field or argument cannot be null.
ORM — Object-relational mapping between application objects and relational data.
Pagination — Dividing a result collection into bounded pages.
Persistence context — JPA unit-of-work context that tracks managed entities and their changes.
Resolver — Code that provides the value for a GraphQL field.
SDL — GraphQL Schema Definition Language.
Transaction — A unit of database work committed or rolled back as one operation.

Project references

The actual schema is in `src/main/resources/graphql/schema.graphqls`. Implementation and test paths are listed in Section 1. The repository's `DEMO_GUIDE.md` contains recorded live rehearsal notes and environment-specific startup details; `SUBMISSION_INDEX.md` is the reviewer entry point. The architecture, requirements, and milestone evidence remain in `ARCHITECTURE.md`, `REQUIREMENTS_AND_DECISIONS.md`, and `TESTING_AND_BACKLOG.md`.